

EC200A-CN(TA) QuecOpen

数据拨号 API 参考手册

LTE Standard 模块系列

版本：1.0.0

日期：2023-04-11

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2023，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2023.

文档历史

修订记录

版本	日期	作者	变更表述
-	2021-12-10	Ryder WANG	文档创建
1.0.0	2023-04-11	Ryder WANG/ Allen FENG	临时版本

目录

文档历史	3
目录	4
表格索引	6
图片索引	7
1 引言	8
2 数据拨号	9
2.1. 拨号流程	9
2.2. 重连机制	10
2.3. 拨号状态	10
2.4. 异常处理	11
3 数据拨号 API	12
3.1. 头文件	12
3.2. 函数概览	12
3.3. 函数详解	14
3.3.1. ql_data_call_init	14
3.3.2. ql_data_call_create	14
3.3.3. ql_data_call_param_alloc	15
3.3.3.1. ql_data_call_param_t	15
3.3.4. ql_data_call_param_init	15
3.3.5. ql_data_call_param_free	16
3.3.6. ql_data_call_param_set_apn_id	16
3.3.7. ql_data_call_param_get_apn_id	17
3.3.8. ql_data_call_param_set_apn_name	17
3.3.9. ql_data_call_param_get_apn_name	18
3.3.10. ql_data_call_param_set_user_name	18
3.3.11. ql_data_call_param_get_user_name	19
3.3.12. ql_data_call_param_set_user_password	19
3.3.13. ql_data_call_param_get_user_password	20
3.3.14. ql_data_call_param_set_auth_pref	20
3.3.15. ql_data_call_param_get_auth_pref	21
3.3.16. ql_data_call_param_set_ip_version	21
3.3.16.1. QL_NET_IP_VER_E	22
3.3.17. ql_data_call_param_get_ip_version	22
3.3.18. ql_data_call_param_set_reconnect_mode	23
3.3.18.1. QL_NET_DATA_CALL_RECONNECT_MODE_E	23
3.3.19. ql_data_call_param_get_reconnect_mode	24
3.3.20. ql_data_call_param_set_reconnect_interval	24
3.3.21. ql_data_call_param_get_reconnect_interval	25
3.3.22. ql_data_call_config	25
3.3.23. ql_data_call_get_config	26

3.3.24.	ql_data_call_start	26
3.3.25.	ql_data_call_stop	27
3.3.26.	ql_data_call_delete	27
3.3.27.	ql_data_call_get_list.....	28
3.3.27.1.	ql_data_call_item_t	28
3.3.28.	ql_data_call_get_status	29
3.3.28.1.	ql_data_call_status_t	29
3.3.28.2.	QL_NET_DATA_CALL_STATUS_E.....	30
3.3.28.3.	ql_net_addr_t	31
3.3.28.4.	ql_net_addr6_t	32
3.3.29.	ql_data_call_set_status_ind_cb.....	33
3.3.29.1.	ql_data_call_status_ind_cb_f.....	33
3.3.30.	ql_data_call_set_apn_config	34
3.3.30.1.	ql_data_call_apn_config_t	34
3.3.30.2.	QL_NET_AUTH_PREF_E	35
3.3.31.	ql_data_call_get_apn_config	35
3.3.32.	ql_data_call_set_service_error_cb	36
3.3.32.1.	ql_data_call_service_error_cb_f.....	36
3.3.33.	ql_data_call_deinit	37
3.4.	函数使用示例.....	37
3.5.	注意事项	39
4	附录 参考文档及术语缩写	40

表格索引

表 1: 函数概览	12
表 2: 参考文档	40
表 3: 术语和缩写	40

Preliminary Document
Not Checked

图片索引

图 1：拨号示意图	9
图 2：数据拨号状态转换	11

Preliminary Document
Not Checked

1 引言

移远通信 LTE Standard EC200A-CN(TA)模块支持 QuecOpen®方案：QuecOpen®是开源的基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

无线数据业务的建立过程，即称为数据拨号（Data Call）。设备携带 APN 信息向运营商网络侧发起数据拨号，运营商网络侧根据 APN 信息建立数据链路到相应网络。本文档介绍数据拨号的过程以及 SDK 中用于拨号的 API。

2 数据拨号

2.1. 拨号流程

运营商在签约时为(U)SIM 卡分配一个或多个 APN 信息，常见的是一路公网 APN 信息以及一路或者多路私网 APN 信息。模块携带 APN 信息进行数据拨号时，运营商网络侧根据 APN 信息分配 IP 地址以及网络，具体流程可参考图1。

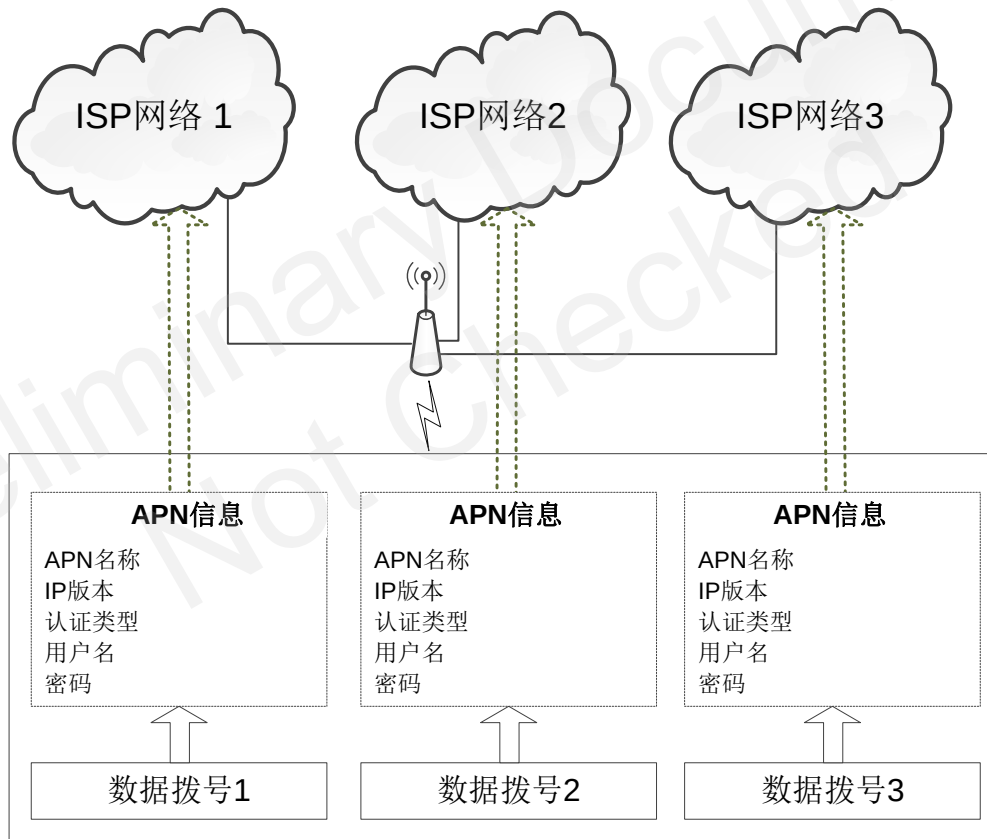


图 1：拨号示意图

数据拨号成功的前提是模块已注册到运营商网络。LTE 网络注册过程中，需建立默认承载。和数据拨号类似，建立默认承载需要传入 APN 信息，若传入的 APN 信息错误，可能导致网络注册失败；运营商分配的 APN 信息中，有一路 APN 信息可用来建立默认承载，详情需咨询运营商。

在发起数据拨号时，模块不会判断是否已注册网络，所以在网络未注册时发起数据拨号会失败，这是正常现象。请参考文档 [2] 来注册网络，再进行数据拨号。

模块首次检测到(U)SIM 卡时，会根据(U)SIM 卡的 PLMN 信息默认加载一组常用的 APN 信息，可通过 **AT+CGDCONT?**（请参考文档 [3]）查看。模块最多支持 4 路 APN，其中 APN1 用于建立默认承载，数据拨号推荐使用 APN2~APN4，每路数据拨号对应一路 APN。

推荐的拨号流程如下：

步骤 1： 设置默认承载 APN 信息（APN1）；

步骤 2： 设置拨号使用的 APN 信息，配置 APN 名称、IP 类型等；

步骤 3： 使用步骤 2 配置的 APN，进行数据拨号。

2.2. 重连机制

拨号失败或拨号断开后，可通过 `ql_data_call_param_set_reconnect_mode()` 设置自动重连模式并配合 `ql_data_call_param_set_reconnect_interval()` 设置重试间隔时间，请参考第 3.3.18 章和第 3.3.20 章。

- **DISABLE**

不自动重连，可按需进行手动重新拨号。

- **NORMAL**

根据设置的时间间隔进行重连，建议设置 20 秒以上，频繁拨号可能会被运营商加入黑名单。

- **MODE1**

以时间退避模式进行重连，例如时间间隔列表为 T1、T2、T3.....Tn，重连的时间间隔为 T1、T2、T3.....Tn、Tn、Tn.....Tn 直至拨号成功；若需要再次重连，则间隔时间从 T1 开始。

- **MODE2**

循环时间退避模式进行重连，例如时间间隔列表为 T1、T2、T3.....Tn，重连的时间间隔为 T1、T2、T3...Tn、T1、T2、T3...Tn、T1、T2、T3...Tn...直至拨号成功；若需要再次重连，则间隔时间从 T1 开始。

2.3. 拨号状态

数据拨号过程包括多种状态，其中 **CONNECTED** 为拨号成功状态，在此状态下可获取 IP 地址等信息；**DISCONNECTED** 为拨号失败或者拨号断开，在此状态下可获取拨号错误码。拨号错误码定义在头文件 `ql-sysroots/usr/include/qmi/wireless_data_service_v01.h` 下。

数据拨号过程含以下状态：

- NONE**：数据拨号实例不存在；
- CREATE**：数据拨号实例被创建；
- IDLE**：数据拨号实例已经被配置；
- CONNECTING**：正在进行拨号；
- CONNECTED**：数据拨号成功，获取到 IP 地址等信息；
- DISCONNECTED**：数据拨号失败或者断开；
- DELETE**：数据拨号实例被删除。

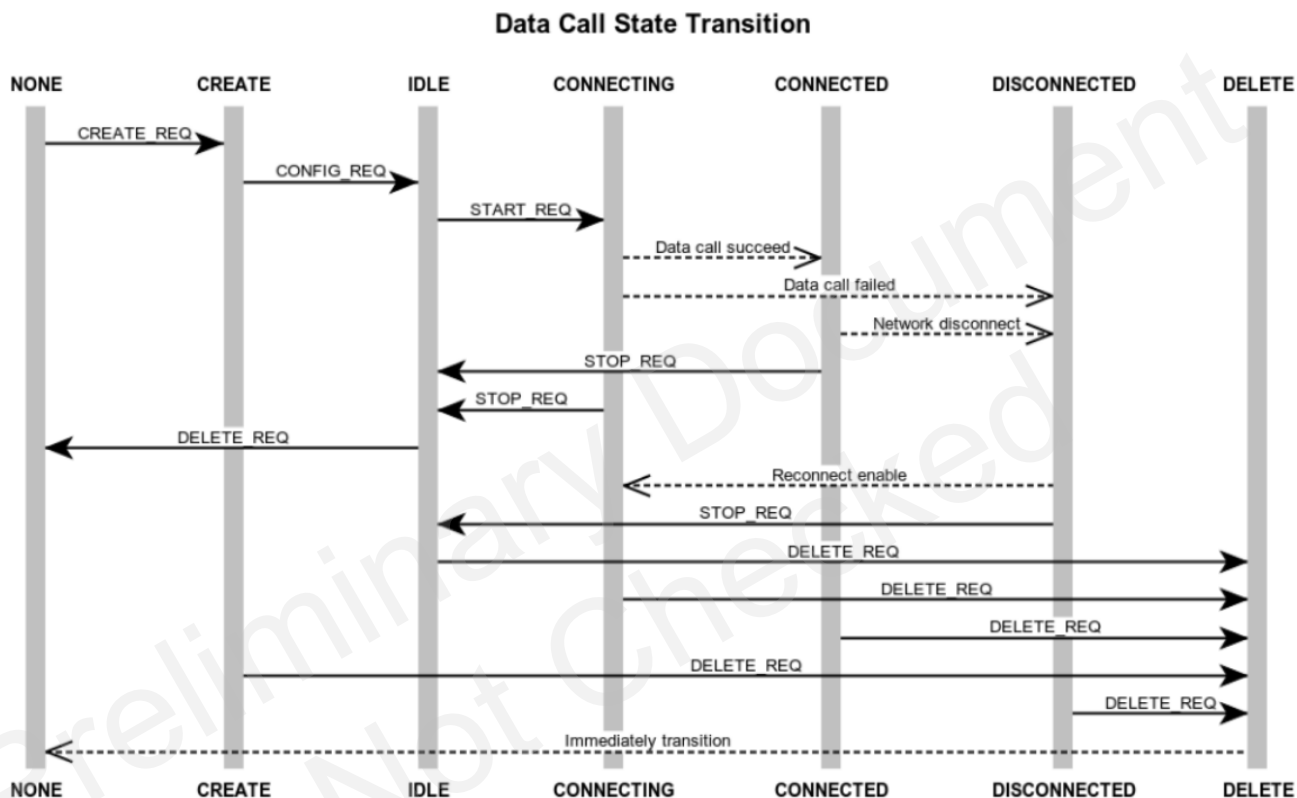


图 2：数据拨号状态转换

2.4. 异常处理

数据拨号 API 接口使用 C/S（Client/Server）方式实现。API 接口在初始化时，创建与服务程序的通信连接。在进行 API 接口调用时，API 接口发送请求给服务程序，服务程序处理完成后返回结果给 API 接口。如果服务程序出现异常退出，调用 API 接口时会返回 `QL_ERR_ABORTED`；或者，用户可以调用 `ql_data_call_set_service_error_cb()` 注册服务异常的回调函数，用以监控服务程序的异常退出。在检测到服务异常退出后，用户可以先调用 `ql_data_call_deinit()` 注销接口，然后再调用 `ql_data_call_init()` 重新启动数据拨号服务，进行业务恢复。

3 数据拨号 API

3.1. 头文件

API 头文件位于 `ql-sysroots/usr/include/ql-sdk` 目录下，其中数据拨号 API 头文件为 `ql_data_call.h`，部分公用结构体定义在头文件 `ql_net_common.h` 中。

3.2. 函数概览

表 1：函数概览

函数	描述
<code>ql_data_call_init()</code>	初始化数据拨号服务
<code>ql_data_call_create()</code>	创建数据拨号实例
<code>ql_data_call_param_alloc()</code>	申请数据拨号配置实例
<code>ql_data_call_param_init()</code>	初始化数据拨号配置实例
<code>ql_data_call_param_free()</code>	释放数据拨号配置实例
<code>ql_data_call_param_set_apn_id()</code>	设置数据拨号配置实例的 APN ID
<code>ql_data_call_param_get_apn_id()</code>	获取数据拨号配置实例的 APN ID
<code>ql_data_call_param_set_apn_name()</code>	设置数据拨号配置实例的 APN 名称
<code>ql_data_call_param_get_apn_name()</code>	获取数据拨号配置实例的 APN 名称
<code>ql_data_call_param_set_user_name()</code>	设置数据拨号配置实例的用户名
<code>ql_data_call_param_get_user_name()</code>	获取数据拨号配置实例的用户名
<code>ql_data_call_param_set_user_password()</code>	设置数据拨号配置实例的用户密码
<code>ql_data_call_param_get_user_password()</code>	获取数据拨号配置实例的用户密码

<code>ql_data_call_param_set_auth_pref()</code>	设置数据拨号配置实例的认证类型
<code>ql_data_call_param_get_auth_pref()</code>	获取数据拨号配置实例的认证类型
<code>ql_data_call_param_set_ip_version()</code>	设置数据拨号配置实例的 IP 类型
<code>ql_data_call_param_get_ip_version()</code>	获取数据拨号配置实例的 IP 类型
<code>ql_data_call_param_set_reconnect_mode()</code>	设置自动重连模式
<code>ql_data_call_param_get_reconnect_mode()</code>	获取自动重连模式配置
<code>ql_data_call_param_set_reconnect_interval()</code>	设置自动重连间隔
<code>ql_data_call_param_get_reconnect_interval()</code>	获取自动重连间隔配置
<code>ql_data_call_config()</code>	设置配置实例到数据拨号实例
<code>ql_data_call_get_config()</code>	获取数据拨号实例的配置实例
<code>ql_data_call_start()</code>	开始数据拨号
<code>ql_data_call_stop()</code>	停止数据拨号
<code>ql_data_call_delete()</code>	删除数据拨号实例
<code>ql_data_call_get_list()</code>	获取数据拨号实例列表
<code>ql_data_call_get_status()</code>	获取数据拨号状态
<code>ql_data_call_set_status_ind_cb()</code>	设置数据拨号状态变化上报回调函数
<code>ql_data_call_set_apn_config()</code>	设置 APN 配置信息
<code>ql_data_call_get_apn_config()</code>	获取 APN 配置信息
<code>ql_data_call_set_service_error_cb()</code>	设置服务程序异常退出上报的回调函数
<code>ql_data_call_deinit()</code>	去初始化数据拨号服务

3.3. 函数详解

3.3.1. ql_data_call_init

该函数用于初始化数据拨号服务。

- 函数原型

```
int ql_data_call_init(void)
```

- 参数

无

- 返回值

QL_ERR_OK

函数执行成功

QL_ERR_SERVICE_NOT_READY

数据拨号服务未准备就绪，建议重试；

其他值

函数执行失败，错误码详见 *ql_type.h*。

备注

使用其他数据拨号 API 前，必须调用该函数初始化数据拨号服务。

3.3.2. ql_data_call_create

该函数用于创建数据拨号实例。

- 函数原型

```
int ql_data_call_create(int call_id, const char *call_name, int is_background)
```

- 参数

call_id:

[In] 数据拨号实例的唯一标识，范围：1~INT MAX，需自定义。

call_name:

[In] 数据拨号实例名称，需自定义。

is_background:

[In] 设置数据拨号实例是否由后台服务管理。

- 1 数据拨号由后台服务管理，即使拨号程序退出，拨号实例仍然存在；
- 0 数据拨号由拨号程序管理，若拨号程序退出，拨号实例也被相应删除。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_SERVICE_NOT_READY	数据拨号服务未准备就绪，建议重试；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.3. ql_data_call_param_alloc

该函数用于申请数据拨号配置实例。此配置实例使用完成后，应调用 *ql_data_call_param_free()* 释放配置实例。

- 函数原型

```
ql_data_call_param_t *ql_data_call_param_alloc(void)
```

- 参数

无

- 返回值

NULL	内存不足，函数执行失败；
配置实例的地址	函数执行成功

3.3.3.1. ql_data_call_param_t

数据拨号配置实例的类型定义如下，用户无需关注其格式及内容。

```
typedef void ql_data_call_param_t
```

3.3.4. ql_data_call_param_init

该函数用于初始化数据拨号配置实例。

- 函数原型

```
int ql_data_call_param_init(ql_data_call_param_t *param)
```

- 参数

param:

[In] 需要初始化的数据拨号配置实例，由 *ql_data_call_param_alloc()* 申请。

- 返回值

QL_ERR_OK 函数执行成功
QL_ERR_INVALID_ARG 非法参数，函数执行失败。

3.3.5. ql_data_call_param_free

该函数用于释放数据拨号配置实例。

- 函数原型

```
int ql_data_call_param_free(ql_data_call_param_t *param)
```

- 参数

param:

[In] 需要释放的数据拨号配置实例，由 *ql_data_call_param_alloc()* 申请。

- 返回值

QL_ERR_OK 函数执行成功
QL_ERR_INVALID_ARG 非法参数，函数执行失败。

3.3.6. ql_data_call_param_set_apn_id

该函数用于设置数据拨号配置实例的 APN ID。

- 函数原型

```
int ql_data_call_param_set_apn_id(ql_data_call_param_t *param, int apn_id)
```

- 参数

param:

[In] 数据拨号配置实例，由 *ql_data_call_param_alloc()* 申请。

apn_id:

[In] 为数据拨号配置实例设置的 APN ID，范围：1~4，数据拨号建议使用 2~4。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.7. ql_data_call_param_get_apn_id

该函数用于获取数据拨号配置实例的 APN ID。

- 函数原型

```
int ql_data_call_param_get_apn_id(ql_data_call_param_t *param, int *apn_id)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

apn_id:

[Out] 指定数据拨号配置实例的 APN ID。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.8. ql_data_call_param_set_apn_name

该函数用于设置数据拨号配置实例的 APN 名称。

- 函数原型

```
int ql_data_call_param_set_apn_name(ql_data_call_param_t *param, const char *apn_name)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

apn_name:

[In] 为数据拨号配置实例设置的 APN 名称。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.9. ql_data_call_param_get_apn_name

该函数用于获取数据拨号配置实例的 APN 名称。

- 函数原型

```
int ql_data_call_param_get_apn_name(ql_data_call_param_t *param, char *buf, int buf_len)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

buf:

[Out] APN 名称缓存。

buf_len:

[In] APN 名称缓存长度。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.10. ql_data_call_param_set_user_name

该函数用于设置数据拨号配置实例的用户名。

- 函数原型

```
int ql_data_call_param_set_user_name(ql_data_call_param_t *param, const char *user_name)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

user_name:

[In] 为数据拨号配置实例设置的用户名。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.11. ql_data_call_param_get_user_name

该函数用于获取数据拨号配置实例的用户名。

- 函数原型

```
int ql_data_call_param_get_user_name(ql_data_call_param_t *param, char *buf, int buf_len)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

buf:

[Out] APN 用户名缓存。

buf_len:

[In] APN 用户名缓存长度。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.12. ql_data_call_param_set_user_password

该函数用于设置数据拨号配置实例的用户密码。

- 函数原型

```
int ql_data_call_param_set_user_password(ql_data_call_param_t *param, const char *user_password)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

user_password:

[In] 指定数据拨号配置实例的 APN 用户密码。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.13. ql_data_call_param_get_user_password

该函数用于获取数据拨号配置实例的用户密码。

- 函数原型

```
int ql_data_call_param_get_user_password(ql_data_call_param_t *param, char *buf, int buf_len)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

buf:

[Out] APN 用户密码缓存。

buf_len:

[In] APN 用户密码缓存长度。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.14. ql_data_call_param_set_auth_pref

该函数用于设置数据拨号配置实例的认证类型。

- 函数原型

```
int ql_data_call_param_set_auth_pref(ql_data_call_param_t *param, int auth_pref)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

auth_pref:

[In] 为数据拨号配置实例设置的认证类型。认证类型的枚举详见第 3.3.30.2 章。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.15. ql_data_call_param_get_auth_pref

该函数用于获取数据拨号配置实例的认证类型。

- 函数原型

```
int ql_data_call_param_get_auth_pref(ql_data_call_param_t *param, int *p_data)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

p_data:

[Out] 认证类型。认证类型的枚举详见第 3.3.30.2 章。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.16. ql_data_call_param_set_ip_version

该函数用于设置数据拨号配置实例的 IP 类型。

- 函数原型

```
int ql_data_call_param_set_ip_version(ql_data_call_param_t *param, int ip_ver)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

ip_ver:

[In] IP 类型，`QL_NET_IP_VER_E` 枚举类型详见第 3.3.16.1 章。

- 返回值

QL_ERR_OK 函数执行成功
QL_ERR_INVALID_ARG 非法参数，函数执行失败。

3.3.16.1. QL_NET_IP_VER_E

IP 类型枚举定义如下：

```
typedef enum {
    QL_NET_IP_VER_MIN = -0,
    QL_NET_IP_VER_V4 = 0x1,
    QL_NET_IP_VER_V6 = 0x2,
    QL_NET_IP_VER_V4V6 = 0x3,
    QL_NET_IP_VER_MAX
} QL_NET_IP_VER_E
```

- 成员

成员	描述
QL_NET_IP_VER_V4	IPv4
QL_NET_IP_VER_V6	IPv6
QL_NET_IP_VER_V4V6	IPv4v6（暂不支持）

3.3.17. ql_data_call_param_get_ip_version

该函数用于获取数据拨号配置实例的 IP 类型。

- 函数原型

```
int ql_data_call_param_get_ip_version(ql_data_call_param_t *param, int *ip_ver)
```

- 参数

param:

[In] 数据拨号配置实例，由 *ql_data_call_param_alloc()* 申请。

ip_ver:

[Out] 指定数据拨号配置实例的 IP 类型，QL_NET_IP_VER_E 枚举类型详见第 3.3.16.1 章。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.18. ql_data_call_param_set_reconnect_mode

该函数用于设置自动重连模式，与函数 `ql_data_call_param_set_reconnect_interval()` 配合使用。

- 函数原型

```
int ql_data_call_param_set_reconnect_mode(ql_data_call_param_t *param, int mode)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

mode:

[In] 重连模式，请参考第 2.2 章，`QL_NET_DATA_CALL_RECONNECT_MODE_E` 枚举类型详见第 3.3.18.1 章。

- 返回值

<code>QL_ERR_OK</code>	函数执行成功
<code>QL_ERR_INVALID_ARG</code>	非法参数，函数执行失败。

3.3.18.1. QL_NET_DATA_CALL_RECONNECT_MODE_E

重连机制的枚举定义如下。有关重连机制的详细介绍详见第 2.2 章。

```
typedef enum {
    QL_NET_DATA_CALL_RECONNECT_MODE_MIN = -1,
    QL_NET_DATA_CALL_RECONNECT_DISABLE = 0x0,
    QL_NET_DATA_CALL_RECONNECT_NORMAL = 0x1,
    QL_NET_DATA_CALL_RECONNECT_MODE_1 = 0x2,
    QL_NET_DATA_CALL_RECONNECT_MODE_2 = 0x3,
    QL_NET_DATA_CALL_RECONONECT_MODE_MAX
} QL_NET_DATA_CALL_RECONNECT_MODE_E
```

- 成员

成员	描述
<code>QL_NET_DATA_CALL_RECONNECT_DISABLE</code>	DISABLE
<code>QL_NET_DATA_CALL_RECONNECT_NORMAL</code>	NORMAL
<code>QL_NET_DATA_CALL_RECONNECT_MODE_1</code>	MODE1
<code>QL_NET_DATA_CALL_RECONNECT_MODE_2</code>	MODE2

3.3.19. ql_data_call_param_get_reconnect_mode

该函数用于获取为指定数据拨号配置实例设置的重连模式。

- 函数原型

```
int ql_data_call_param_get_reconnect_mode(ql_data_call_param_t *param, int *p_mode)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

p_mode:

[Out] 为指定数据拨号配置实例设置的重连模式，请参考第 2.2 章，`QL_NET_DATA_CALL_RECONNECT_MODE_E` 枚举类型详见第 3.3.18.1 章。

- 返回值

`QL_ERR_OK` 函数执行成功
`QL_ERR_INVALID_ARG` 非法参数，函数执行失败。

3.3.20. ql_data_call_param_set_reconnect_interval

该函数用于设置自动重连间隔，与函数 `ql_data_call_param_set_reconnect_mode()` 配合使用。

- 函数原型

```
int ql_data_call_param_set_reconnect_interval(ql_data_call_param_t *param, int *time_list, int num)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

time_list:

[In] 时间间隔列表，整型数组。数组成员单位：秒。最大支持数组长度为 20。

num:

[In] *time_list* 的数组长度。

- 返回值

`QL_ERR_OK` 函数执行成功

`QL_ERR_INVALID_ARG` 非法参数，函数执行失败。

3.3.21. ql_data_call_param_get_reconnect_interval

该函数用于获取为指定数据拨号配置实例设置的重连间隔。

- 函数原型

```
int ql_data_call_param_get_reconnect_interval(ql_data_call_param_t *param, int *time_list, int *p_num)
```

- 参数

param:

[In] 数据拨号配置实例，由 `ql_data_call_param_alloc()` 申请。

time_list:

[Out] 时间间隔列表，整型数组。数组成员单位：秒。最大支持数组长度为 20。

p_num:

[In/Out] 整型指针，传入时指针内容为 *time_list* 的数组长度，返回时指针内容为数组中的有效成员个数。

- 返回值

`QL_ERR_OK` 函数执行成功

`QL_ERR_INVALID_ARG` 非法参数，函数执行失败。

3.3.22. ql_data_call_config

该函数使用数据拨号配置实例来配置数据拨号实例。

- 函数原型

```
int ql_data_call_config(int call_id, ql_data_call_param_t *param)
```

- 参数

call_id:

[In] 数据拨号实例的唯一标识，范围：1~INT MAX，需自定义。

param:

[In] 数据拨号配置实例，由 *ql_data_call_param_alloc()* 申请。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_INVALID_ARG	非法参数，函数执行失败；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.23. ql_data_call_get_config

该函数用于获取数据拨号实例的配置实例。

- 函数原型

```
int ql_data_call_get_config(int call_id, ql_data_call_param_t *param)
```

- 参数

call_id:

[In] 数据拨号实例的唯一标识，范围：1~INT MAX，需自定义。

param:

[Out] 数据拨号配置实例，由 *ql_data_call_param_alloc()* 申请。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_INVALID_ARG	非法参数，函数执行失败；
其他值	函数执行失败，错误码详见 <i>ql_type.h</i> 。

3.3.24. ql_data_call_start

该函数用于开始数据拨号。

- 函数原型

```
int ql_data_call_start(int call_id)
```

- 参数

call_id:

[In] 数据拨号实例的唯一标识，范围：1~INT MAX，需自定义。

- 返回值

QL_ERR_OK

函数执行成功

QL_ERR_INVALID_ARG

非法参数，函数执行失败；

其他值

函数执行失败，错误码详见 *ql_type.h*。

3.3.25. ql_data_call_stop

该函数用于停止数据拨号。

- 函数原型

```
int ql_data_call_stop(int call_id)
```

- 参数

call_id:

[In] 数据拨号实例的唯一标识，范围：1~INT MAX，需自定义。

- 返回值

QL_ERR_OK

函数执行成功

QL_ERR_INVALID_ARG

非法参数，函数执行失败；

其他值

函数执行失败，错误码详见 *ql_type.h*。

3.3.26. ql_data_call_delete

该函数用于删除数据拨号实例。

- 函数原型

```
int ql_data_call_delete(int call_id)
```

- 参数

call_id:

[In] 数据拨号实例的唯一标识，范围：1~INT MAX，需自定义。

- 返回值

QL_ERR_OK

函数执行成功

QL_ERR_INVALID_ARG

非法参数，函数执行失败；

其他值

函数执行失败，错误码详见 *ql_type.h*。

3.3.27. ql_data_call_get_list

该函数用于获取当前存在的数据拨号实例列表。

- 函数原型

```
int ql_data_call_get_list(ql_data_call_item_t *list, int *list_len)
```

- 参数

list:

[Out] 数据拨号实例列表的数组。结构体定义详见第 3.3.27.1 章。

list_len:

[In/Out] 传入数据拨号实例列表数据长度，返回标识数组中有效的个数。

- 返回值

QL_ERR_OK

函数执行成功

QL_ERR_INVALID_ARG

非法参数，函数执行失败；

其他值

函数执行失败，错误码详见 *ql_type.h*。

3.3.27.1. ql_data_call_item_t

当前存在的数据拨号实例信息结构体定义如下：

```
typedef struct
{
    int call_id;
    char call_name[QL_NET_MAX_NAME_LEN];
} ql_data_call_item_t
```

- 参数

类型	参数	描述
int	<i>call_id</i>	数据拨号的 ID
char	<i>call_name</i>	数据拨号的名称

3.3.28. ql_data_call_get_status

该函数用于获取数据拨号实例的状态。

- 函数原型

```
int ql_data_call_get_status(int call_id, ql_data_call_status_t *p_sta)
```

- 参数

call_id:

[In] 数据拨号实例的唯一标识，范围：1~INT MAX，需自定义。

p_sta:

[Out] 返回数据拨号实例的状态以及 IP 地址等信息。结构体定义详见第 3.3.28.1 章。

- 返回值

QL_ERR_OK

函数执行成功

QL_ERR_INVALID_ARG

非法参数，函数执行失败；

其他值

函数执行失败，错误码详见 *ql_type.h*。

3.3.28.1. ql_data_call_status_t

数据拨号的状态信息结构体定义如下：

```
typedef struct
{
    int call_id;
    char call_name[QL_NET_MAX_NAME_LEN];
    QL_NET_IP_VER_E ip_ver;
    QL_NET_DATA_CALL_STATUS_E call_status;
    char device[QL_NET_MAX_NAME_LEN];
    uint8_t has_addr;
    ql_net_addr_t addr;
    uint8_t has_addr6;
```

```

ql_net_addr6_t addr6;
int call_end_reason_type;
int call_end_reason_code;
} ql_data_call_status_t
    
```

● 参数

类型	参数	描述
int	<i>call_id</i>	数据拨号 ID
char	<i>call_name</i>	数据拨号的名称
<i>QL_NET_IP_VER_E</i>	<i>ip_ver</i>	IP 类型。 枚举详见第 3.3.16.1 章。
<i>QL_NET_DATA_CALL_STATUS_E</i>	<i>call_status</i>	数据拨号的状态。 枚举详见第 3.3.28.2 章。
char	<i>device</i>	数据拨号对应的网卡名
uint8_t	<i>has_addr</i>	是否获取到 IPv4 地址
<i>ql_net_addr_t</i>	<i>addr</i>	IPv4 地址信息。 结构体定义详见第 3.3.28.3 章。
uint8_t	<i>has_addr6</i>	是否获取到 IPv6 地址
<i>ql_net_addr6_t</i>	<i>addr6</i>	IPv6 地址类型。 结构体定义详见第 3.3.28.4 章。
int	<i>call_end_reason_type</i>	上一次数据拨号断开原因类型
int	<i>call_end_reason_code</i>	上一次数据拨号断开的原因码

3.3.28.2. QL_NET_DATA_CALL_STATUS_E

数据拨号状态的枚举定义如下：

```

typedef enum {
    QL_NET_DATA_CALL_STATUS_MIN = -1,
    QL_NET_DATA_CALL_STATUS_NONE = 0x0,
    QL_NET_DATA_CALL_STATUS_CREATED = 0x1,
    QL_NET_DATA_CALL_STATUS_IDLE = 0x2,
    QL_NET_DATA_CALL_STATUS_CONNECTING = 0x3,
    QL_NET_DATA_CALL_STATUS_PARTIAL_V4_CONNECTED = 0x4,
    QL_NET_DATA_CALL_STATUS_PARTIAL_V6_CONNECTED = 0x5,
    QL_NET_DATA_CALL_STATUS_CONNECTED = 0x6,
    QL_NET_DATA_CALL_STATUS_DISCONNECTED = 0x7,
}
    
```

```
QL_NET_DATA_CALL_STATUS_ERROR = 0x8,
QL_NET_DATA_CALL_STATUS_DELETED = 0x9,
QL_NET_DATA_CALL_STATUS_MAX
} QL_NET_DATA_CALL_STATUS_E
```

● 成员

成员	描述
<code>QL_NET_DATA_CALL_STATUS_NONE</code>	置空状态
<code>QL_NET_DATA_CALL_STATUS_CREATED</code>	创建状态
<code>QL_NET_DATA_CALL_STATUS_IDLE</code>	空闲状态
<code>QL_NET_DATA_CALL_STATUS_CONNECTING</code>	连接中状态
<code>QL_NET_DATA_CALL_STATUS_PARTIAL_V4_CONNECTED</code>	IPv4 连接成功, IPv6 未连接
<code>QL_NET_DATA_CALL_STATUS_PARTIAL_V6_CONNECTED</code>	IPv6 连接成功, IPv4 未连接
<code>QL_NET_DATA_CALL_STATUS_CONNECTED</code>	连接成功
<code>QL_NET_DATA_CALL_STATUS_DISCONNECTED</code>	连接断开
<code>QL_NET_DATA_CALL_STATUS_ERROR</code>	数据拨号失败
<code>QL_NET_DATA_CALL_STATUS_DELETED</code>	删除状态

3.3.28.3. ql_net_addr_t

IPv4 类型数据拨号的地址信息的结构体定义如下:

```
typedef struct {
    char addr[QL_NET_MAX_ADDR_LEN];
    char netmask[QL_NET_MAX_ADDR_LEN];
    uint8_t subnet_bits;
    char gateway[QL_NET_MAX_ADDR_LEN];
    char dnsp[QL_NET_MAX_ADDR_LEN];
    char dnss[QL_NET_MAX_ADDR_LEN];
} ql_net_addr_t
```

● 参数

类型	参数	描述
char	<i>addr</i>	IPv4 地址
char	<i>netmask</i>	子网掩码
uint8_t	<i>subnet_bits</i>	子网的网络位的长度
char	<i>gateway</i>	网关地址
char	<i>dnsp</i>	首选 DNS
char	<i>dnss</i>	备选 DNS

3.3.28.4. ql_net_addr6_t

IPv6 类型数据拨号的地址信息的结构体定义如下：

```
typedef struct
{
    char addr[QL_NET_MAX_ADDR_LEN];
    char prefix[QL_NET_MAX_ADDR_LEN];
    int32_t prefix_bits;
    char gateway[QL_NET_MAX_ADDR_LEN];
    char dnsp[QL_NET_MAX_ADDR_LEN];
    char dnss[QL_NET_MAX_ADDR_LEN];
} ql_net_addr6_t
```

● 参数

类型	参数	描述
char	<i>addr</i>	IPv6 地址
char	<i>prefix</i>	子网前缀
int32_t	<i>prefix_bits</i>	子网前缀的长度
char	<i>gateway</i>	网关地址
char	<i>dnsp</i>	首选 DNS
char	<i>dnss</i>	备选 DNS

3.3.29. ql_data_call_set_status_ind_cb

该函数用于设置数据拨号实例状态变化上报的回调函数。当数据拨号的状态发生变化，会调用传入的回调函数。

- 函数原型

```
int ql_data_call_set_status_ind_cb(ql_data_call_status_ind_cb_f cb)
```

- 参数

cb:

[In] 传入的回调函数，数据拨号实例的状态发生变化时会调用该回调函数。回调函数详见第3.3.29.1章。

- 返回值

QL_ERR_OK	函数执行成功
QL_ERR_INVALID_ARG	非法参数，函数执行失败。

备注

回调函数中不可调用任何数据拨号接口，且在回调函数中禁止处理耗时或者阻塞的任务。

3.3.29.1. ql_data_call_status_ind_cb_f

数据拨号实例状态变化的回调函数定义如下：

```
typedef void (*ql_data_call_status_ind_cb_f)(int call_id,  
      QL_NET_DATA_CALL_STATUS_E pre_call_status,  
      ql_data_call_status_t *p_msg)
```

- 参数

call_id:

[In] 数据拨号 ID。

pre_call_status:

[In] 之前数据拨号的状态，枚举类型详见第3.3.28.2章。

p_msg:

[In] 数据拨号的状态信息。

- 返回值

无

3.3.30. ql_data_call_set_apn_config

该函数用于设置 APN 信息。若指定的 APN ID 不存在，则自动创建新 APN，然后再进行 APN 信息配置。

- 函数原型

```
int ql_data_call_set_apn_config(int apn_id, ql_data_call_apn_config_t *p_info)
```

- 参数

apn_id:

[In] APN ID，范围：1~4。APN1 用于建立默认承载，建议使用 APN2~APN4 进行数据拨号。

p_info:

[In] APN 配置信息，结构体定义详见第 3.3.30.1 章。

- 返回值

QL_ERR_OK

函数执行成功

QL_ERR_INVALID_ARG

非法参数，函数执行失败。

3.3.30.1. ql_data_call_apn_config_t

APN 配置信息结构体定义如下：

```
typedef struct
{
    QL_NET_IP_VER_E ip_ver;
    QL_NET_AUTH_PREF_E auth_pref;
    char apn_name[QL_NET_MAX_APN_NAME_LEN];
    char username[QL_NET_MAX_APN_USERNAME_LEN];
    char password[QL_NET_MAX_APN_PASSWORD_LEN];
} ql_data_call_apn_config_t
```

- 参数

类型	参数	描述
QL_NET_IP_VER_E	ip_ver	IP 类型，枚举详见第 3.3.16.1 章。

QL_NET_AUTH_PREF_E	auth_pref	认证类型，枚举详见第 3.3.30.2 章。
char	apn_name	APN 名称
char	username	用户名
char	password	密码

3.3.30.2. QL_NET_AUTH_PREF_E

认证类型的枚举定义如下：

```
typedef enum {
    QL_NET_AUTH_PREF_MIN = -1,
    QL_NET_AUTH_PREF_PAP_CHAP_NOT_ALLOWED = 0,
    QL_NET_AUTH_PREF_PAP_ONLY_ALLOWED = 1,
    QL_NET_AUTH_PREF_CHAP_ONLY_ALLOWED = 2,
    QL_NET_AUTH_PREF_PAP_CHAP_BOTH_ALLOWED = 3,
    QL_NET_AUTH_PREF_MAX
} QL_NET_AUTH_PREF_E
```

● 成员

成员	描述
QL_NET_AUTH_PREF_PAP_CHAP_NOT_ALLOWED	不使用 PAP 和 CHAP 认证
QL_NET_AUTH_PREF_PAP_ONLY_ALLOWED	只使用 PAP 认证
QL_NET_AUTH_PREF_CHAP_ONLY_ALLOWED	只使用 CHAP 认证
QL_NET_AUTH_PREF_PAP_CHAP_BOTH_ALLOWED	使用 PAP 或 CHAP 认证

3.3.31. ql_data_call_get_apn_config

该函数用于获取 APN 信息。

● 函数原型

```
int ql_data_call_get_apn_config(int apn_id, ql_data_call_apn_config_t *p_info)
```

- 参数

apn_id:

[In] APN ID，新描述：范围：1~4。APN1 用于建立默认承载，建议使用 APN2~APN4 进行数据拨号。

p_info:

[Out] APN 配置信息。

- 返回值

QL_ERR_OK

函数执行成功

QL_ERR_INVALID_ARG

非法参数，函数执行失败。

3.3.32. ql_data_call_set_service_error_cb

该函数用于设置数据拨号服务程序异常退出上报的回调函数。

- 函数原型

```
int ql_data_call_set_service_error_cb(ql_data_call_service_error_cb_f cb)
```

- 参数

cb:

[In] 回调函数，如果服务程序发生异常退出，该回调函数会被执行。回调函数详见第 3.3.32.1 章。

- 返回值

QL_ERR_OK

函数执行成功

其他值

函数执行失败，错误码详见 *ql_type.h*。

备注

回调函数中不可调用任何数据拨号接口，且在回调函数中禁止处理耗时或者阻塞的任务。

3.3.32.1. ql_data_call_service_error_cb_f

数据拨号服务异常上报的回调函数定义如下：

```
typedef void (*ql_data_call_service_error_cb_f)(int error)
```

- 参数

error:

[In] 错误码，当前仅返回 `QL_ERR_ABORTED`

- 返回值

无

3.3.33. ql_data_call_deinit

该函数用于去初始化数据拨号服务。

- 函数原型

```
int ql_data_call_deinit(void)
```

- 参数

无

- 返回值

`QL_ERR_OK`

函数执行成功

其他值

函数执行失败，错误码详见 `ql_type.h`。

3.4. 函数使用示例

调用数据拨号函数需了解如何进行 Linux 系统的网络配置以及 DNS 处理。

本章节示例中(U)SIM 卡和运营商签约的 APN 信息为：

- 公网 APN: `apnpublic`
- 私网 APN: `apnprivate`

其中，私网 APN 用于建立默认承载。

本章节介绍如何使用数据拨号接口同时拨两路数据拨号。示例代码参考 `sample/data_call/sample_multi_data_call.c`，伪代码示例如下：

```
data_call_cb()
{
    /* 拨号状态处理 */
}
```

```

main()
{
    /* 初始化拨号服务 */
    ql_data_call_init

    /* 设置拨号状态变化上报回调函数 */
    ql_data_call_set_status_ind_cb data_call_cb

    /* 设置默认承载 APN 的信息 */
    ql_data_call_set_apn_config apn_id=1, apn_name=apnprivate, ip_ver=QL_NET_IP_VER_V4

    /* 公网拨号使用 APN2, 设置 APN2 信息 */
    ql_data_call_set_apn_config apn_id=2, apn_name= apnpublic, ip_ver=ipv4

    /* 私网拨号使用 APN3, 设置 APN3 信息 */
    ql_data_call_set_apn_config apn_id=3, apn_name= apnprivate, ip_ver=ipv4

    /* 创建公网数据拨号实例, 并开始拨号, call_id 为 1 */
    ql_data_call_create call_id=1, call_name=public, is_background=0
    /* 配置公网拨号的配置实例 */
    ql_data_call_param_set_apn_id apn_id=2
    ql_data_call_param_set_ip_version ip_ver=ipv4
    ql_data_call_param_set_reconnect_mode
    ql_data_call_param_set_reconnect_interval

    /* 设置配置实例到数据拨号实例 */
    ql_data_call_config call_id=1
    /* 开始拨号 */
    ql_data_call_start call_id=1

    /* 创建私网拨号实例, 并开始拨号, call_id 为 2 */
    ql_data_call_create call_id=2, call_name=private, is_background=0
    /* 配置私网拨号的配置实例 */
    ql_data_call_param_set_apn_id apn_id=3
    ql_data_call_param_set_ip_version ip_ver=ipv4
    ql_data_call_param_set_reconnect_mode
    ql_data_call_param_set_reconnect_interval

    /* 设置配置实例到数据拨号实例 */
    ql_data_call_config call_id=2
    /* 开始拨号 */
    ql_data_call_start call_id=2
}

```

3.5. 注意事项

1. 不可在数据拨号状态回调函数中调用其他 API 接口。
2. 不可在数据拨号状态回调函数中处理阻塞事务。
3. 所有数据拨号函数都不支持多线程调用。

Preliminary Document
Not Checked

4 附录 参考文档及术语缩写

表 2：参考文档

文档名称
[1] Quectel_EC200A-CN(TA)_QuecOpen_快速开发指导
[2] Quectel_EC200A-CN(TA)_QuecOpen_蜂窝网络信息 API_参考手册
[3] Quectel_EC200A-CN(TA)_QuecOpen_AT_Commands_Manual

表 3：术语和缩写

缩写	英文描述	中文描述
API	Application Programming Interface	应用程序接口
APN	Access Point Name	接入点名称
DNS	Domain Name System	域名系统
ID	Identifier	标识符
IMS	IP Multimedia Subsystem	IP 多媒体系统
IoT	Internet of Things	物联网
IP	Internet Protocol	网际互连协议
IPv4	Internet Protocol version 4	第 4 版互联网协议
IPv6	Internet Protocol version 6	第 6 版互联网协议
LTE	Long Term Evolution	长期演进
PLMN	Public Land Mobile Network	公共陆地移动（通信）网络
SDK	Software Development Kit	软件开发工具包
(U)SIM	(Universal) Subscriber Identity Module	（通用）用户身份识别模块